

Before start, make sure you've read the following articles:

- **Packages, variables, and functions(full section)**
- **Tour of Go. Read from "For" section until "Exercise: Loops and Functions"**

Flow control statements: for, if, else, switch and defer

For

For continued

For is Go's "while"

Forever

If

If with a short statement

If and else

[Exercise: Loops and Functions](#)

- **More types: structs, slices, and maps**
- **Arrays and Slices**
- **Solving slices problems in Go**

Part 1: Arrays vs slices

Arrays

The expression

```
var a [10]int
```

declares a variable `a` as an array of ten integers.

Task 1:

1. According to the tour of Go, declare an array of 5 float numbers and print them.
2. What will be the output if you initialize a 5-element array with only 3 values? Why?

3. Declare a simple 2D array.

Participation Task: Demonstrate the declaration of 2D array using `var` keyword and `for range` loop.

Slices

An array has a fixed size. A slice, on the other hand, is a dynamically-sized, flexible view into the elements of an array. In practice, slices are much more common than arrays.

The type `[]T` is a slice with elements of type `T`.

Let's consider the slice structure:

```
type slice struct {  
    array unsafe.Pointer  
    len    int  
    cap    int  
}
```

Read the materials and find out what are len and cap.

Look at the example below. Run the code yourself:

```
func main() {  
    cars := []string{"Ferrari", "Honda", "Ford", "BYD"}  
    fmt.Println("cars:", cars, "has old length", len(cars), "and  
capacity", cap(cars)) //capacity of cars is 4  
    cars = append(cars, "Toyota")  
    fmt.Println("cars:", cars, "has new length", len(cars), "and  
capacity", cap(cars)) // What is the len and capacity of cars now? Why?  
}
```

Copying Slices: Avoiding Shared Data Pitfalls

Slices share their underlying array, which can lead to unexpected behavior. The `copy` function creates a new slice with its own array, copying elements from the source.

Here's an example showing the difference:

```

package main

import "fmt"

func main() {
    src := []int{1, 2, 3, 4}
    dst := make([]int, 2)

    // Copy first two elements
    n := copy(dst, src)
    fmt.Println("Copied:", dst, "Elements copied:", n)

    // Modify source
    src[0] = 99
    fmt.Println("Source after mod:", src)
    fmt.Println("Dest after mod:", dst)
}

```

Output:

```

Copied: [1 2] Elements copied: 2
Source after mod: [99 2 3 4]
Dest after mod: [1 2]

```

Task 2:

1. According to the tour of Go and other materials, declare a slice in 4 different ways.
2. Demonstrate modifications done to the slice that affects the underlying array and vice versa.
3. Answer the theoretical questions about arrays & slices(in-person with Bakytzhan-agai):
 - What is the array. What is the slice?
 - What's the difference between array and slice?
 - What is the output of the following code. Why?
 - etc.

| Participation Task:

Write a Go program that passes a slice to a function and modifies its elements inside that function.

Answer the following:

1. What happens to the original slice in `main()` after the function executes?

Defence will be conducted in the next practice lesson, according to your schedule. In-person with your laptops.

